



eSLAM: An Energy-Efficient Accelerator for Real-Time ORB-SLAM on FPGA Platform*

Runze Liu ^{†¶}, Jianlei Yang ^{†¶}, Yiran Chen [§], Weisheng Zhao ^{‡¶}

[†] School of Computer Science and Engineering, Beihang University, Beijing, 100191, China.

[‡] School of Electronic and Information Engineering, Beihang University, Beijing, 100191, China.

[¶] Beijing Advanced Innovation Center for Big Data and Brain Computing, Beihang University, Beijing, 100191, China.

[§] Department of Electrical and Computer Engineering, Duke University, Durham, NC 27708, USA.

jianlei@buaa.edu.cn

weisheng.zhao@buaa.edu.cn

ABSTRACT

Simultaneous Localization and Mapping (SLAM) is a critical task for autonomous navigation. However, due to the computational complexity of SLAM algorithms, it is very difficult to achieve real-time implementation on low-power platforms. We propose an energy-efficient architecture for real-time ORB (Oriented-FAST and Rotated-BRIEF) based visual SLAM system by accelerating the most time-consuming stages of feature extraction and matching on FPGA platform. Moreover, the original ORB descriptor pattern is reformed as a rotational symmetric manner which is much more hardware friendly. Optimizations including rescheduling and parallelizing are further utilized to improve the throughput and reduce the memory footprint. Compared with Intel i7 and ARM Cortex-A9 CPUs on TUM dataset, our FPGA realization achieves up to 3× and 31× frame rate improvement, as well as up to 71× and 25× energy efficiency improvement, respectively.

KEYWORDS

Visual SLAM, ORB, FPGA, Acceleration

1 INTRODUCTION

Simultaneous Localization and Mapping (SLAM) [3] is a critical technique for autonomous navigation systems to build/update a map of the surrounding environment and estimate their own locations in this map. SLAM is a fundamental problem for higher-level tasks such as path planning and navigation, and widely used in applications such as self-driving cars, robotics, virtual reality and augmented reality.

Recently, feature-based visual SLAM has received particular attention because of its robustness to large motions and illumination changes compared with other visual SLAM approaches such as

optical flow method or direct method. Among feature-based approaches, ORB (Oriented-FAST and Rotated-BRIEF) [8] is the most widely adopted feature because of its high efficiency and robustness. However, the high computational intensity of feature extraction and matching makes it very challenging to run ORB-based visual SLAM on low-power embedded platforms, such as drones and mobile robots, for real-time applications.

Several prior efforts have been made to accelerate visual SLAM on low-power platforms, but no fully integrated ORB-based visual SLAM is proposed on such platforms so far. Feature matching and ORB extraction is accelerated on FPGA for visual SLAM system, respectively in [2] and [4]. A SIFT-feature based SLAM is implemented on FPGA [6] where only matrix computation is accelerated but the most time-consuming part, feature extraction, is not involved. A optical-flow based visual inertial odometry is implemented on ASIC [11], which is relatively less computational intensive but may fail in scenarios with variational illuminations or large motions/displacements, because the basic assumptions of optical flow method are invalid in these scenarios [5].

In this paper, eSLAM is proposed as a heterogeneous architecture of ORB-based visual SLAM system. The most time-consuming procedures of feature extraction and matching are accelerated on FPGA while the remaining tasks including pose estimation, pose optimization and map updating are performed on the host ARM processor. The main contributions of this paper are listed as below:

- A novel ORB-based visual SLAM accelerator is proposed for real-time applications on energy-efficient FPGA platforms.
- A rotationally symmetric ORB descriptor pattern is utilized to make our algorithm much more hardware-friendly.
- Optimization including rescheduling and parallelizing are further exploited to improve the computation throughput.

The remainder of this paper is organized as follows. Section 2 presents the ORB-based visual SLAM framework and the introduced rotationally symmetric descriptor. Section 3 illustrates the detailed architecture of eSLAM. Experimental results are evaluated for the proposed eSLAM in Section 4. Concluding remarks are given in Section 5.

2 ORB-BASED VISUAL SLAM SYSTEM

2.1 ORB-SLAM Framework

The ORB-based visual SLAM system takes RGB-D (RGB and depth) images for mapping and localization. Its framework, as shown in Figure 1, consists of five main procedures: feature extraction, feature matching, pose estimation, pose optimization and map updating. In

*This work was supported in part by the National Natural Science Foundation of China (61602022, 61501013, 61571023, 61521091 and 1157040329), State Key Laboratory of Software Development Environment (SKLSDE-2018ZX-07), National Key Technology Program of China (2017ZX01032101), CCF-Tencent IAGR20180101 and the International Collaboration Project under Grant B16001.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '19, June 2–6, 2019, Las Vegas, NV, USA

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6725-7/19/06...\$15.00

<https://doi.org/10.1145/3316781.3317820>

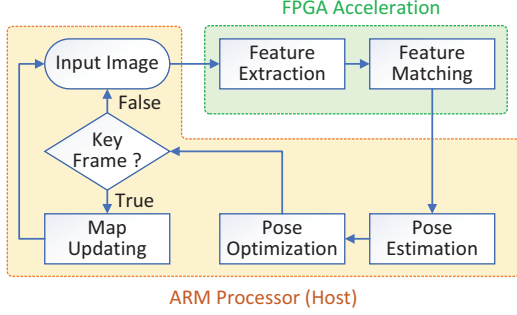


Figure 1: Visual SLAM algorithm framework.

this work, feature extraction and feature matching are accelerated on FPGA, and remaining tasks are performed on ARM processor.

Feature Extraction: In this function, ORB features are extracted from the input RGB images. ORB is a very efficient and robust combination of FAST (Features from Accelerated Segment Test) keypoint and BRIEF (Binary Robust Independent Elementary Features) [1] descriptor. It calculates orientations of every feature and rotates the descriptor pattern accordingly to make the features rotationally invariant. And to obtain scale invariance, a 4-layer pyramid is generated from the original image. Aiming to implement ORB algorithm on hardware efficiently, a hardware-friendly, rotationally symmetric BRIEF descriptor pattern is proposed in this work and illustrated in Section 2.2.

Feature Matching: In feature matching, each feature detected in the current frame is matched with a 3D map point in the global map according to the distance between their BRIEF descriptors. BRIEF descriptors are binary strings, their distances are described by Hamming distances.

Pose Estimation: We apply PnP (Perspective-n-Points) method to the matched feature pairs to estimate the translation and the rotation of the camera. RANSAC (Random Sample Consensus) is used to eliminate the mismatches.

Pose Optimization: In this function, camera pose estimated by PnP is optimized by minimizing the reprojection error of the observed map points. Assuming that the pixel coordinates of the features in the current frame are $(c_1, c_2, \dots, c_n)$, the positions of the matched map points are $(g_1, g_2, \dots, g_n)$, the pose of the camera is p , and $h(g_i, p)$ refers to the pixel coordinate of g_i when it is projected to the current frame. The reprojection error E can be defined as the following formula:

$$E = \sum_{i=1}^n \|c_i - h(g_i, p)\|^2 \quad (1)$$

Levenberg-Marquardt method [7] is applied iteratively to minimize E while adjusting the camera pose p .

Map Updating: Map updating is only executed in key frames. Key frames are a set of frames where the translation or rotation of the camera is larger than a threshold. When a key frame is detected, the 3D map points in the key frame are added to the global map, and the map points that have not been matched for a long period of time are deleted from the global map to prevent it from becoming too large.

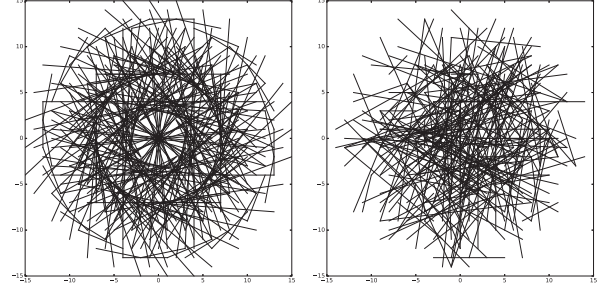


Figure 2: Pattern of RS-BRIEF (left) and BRIEF (right).

2.2 Rotationally Symmetric BRIEF

To compute the BRIEF descriptor of a feature, 2 sets of locations in the neighborhood around the feature, $L_S(S_1, S_2, \dots, S_{256})$ and $L_D(D_1, D_2, \dots, D_{256})$, are introduced. In ORB algorithm, to make the feature invariant to rotation, L_S and L_D are rotated according to the feature's orientation and denoted as $L_{SR}(SR_1, SR_2, \dots, SR_{256})$ and $L_{DR}(DR_1, DR_2, \dots, DR_{256})$. The descriptor $T(B_1, B_2, \dots, B_{256})$ is a 256 bits binary string. B_i is 1 if $I(SR_i) > I(DR_i)$, else it is 0, where $I(SR_i)$ and $I(DR_i)$ are the pixel intensities on location SR_i and DR_i .

Originally, L_S and L_D are randomly selected in the neighborhood according to Gaussian distribution. And every location after rotation needs to be calculated using the following formula:

$$\begin{aligned} x' &= x \cdot \cos\theta - y \cdot \sin\theta \\ y' &= y \cdot \cos\theta + x \cdot \sin\theta \end{aligned} \quad (2)$$

where (x, y) refers to the initial location and (x', y') refers to the location after rotation. Since 512 locations are required to be rotated in order to compute the descriptor of each feature, the rotation procedure is quite compute-intensive.

To reduce the computation cost of rotation procedure, a popular approach is to pre-compute the rotated BRIEF patterns [8] instead of computing them directly each time. In this approach, the orientation of features is discretized into 30 different values, i.e., 12 degrees, 24 degrees, 36 degrees, etc. Then 30 BRIEF patterns after rotation are pre-computed and built as a lookup table. The lookup table is utilized to obtain the descriptors when necessary so that the computation cost could be reduced significantly.

One drawback of the above approach is the degradation in accuracy. Because the orientation of features is discretized, there will be a deviation from the true value which is up to 6 degrees (half of 12 degrees). However, considering that the test locations are selected from a circular patch with a radius of 15 pixels, the maximum error of a test location is about 1 pixel on the smoothed image. Hence, the influence on the accuracy is almost negligible.

Although the pre-computing approach could reduce the computation cost significantly in algorithm level, it is still difficult to implement them on hardware platforms directly. For FPGA hardware implementations, all the 30 BRIEF patterns are required to be pre-computed and stored as a lookup table, which will introduce considerable amount of extra resources so that it still could not satisfy the required energy efficiency.

In order to make descriptor computing more hardware-friendly, we put forward a special way to select the test locations and proposed a 32-fold rotationally symmetric BRIEF pattern (RS-BRIEF). The procedure to generate RS-BRIEF pattern is as follows. First of all,

it selects 2 sets of locations, $L_{S1}(S_1, S_2, \dots, S_8)$ and $L_{D1}(D_1, D_2, \dots, D_8)$, in the neighborhood around the feature according to Gaussian distribution. Each of the 2 sets contains 8 locations. Then, it rotates L_{S1} and L_{D1} by increments of every 11.25 degrees, i.e., 11.25, 22.5, ..., 348.75, to generate $L_{S2}, L_{S3}, \dots, L_{S32}$ and $L_{D2}, L_{D3}, \dots, L_{D32}$. The 2 sets, $L'_S(L_{S1} \cup L_{S2} \cup \dots \cup L_{S32})$ and $L'_D(L_{D1} \cup L_{D2} \cup \dots \cup L_{D32})$, are the final test locations. The RS-BRIEF pattern is visualized and compared with the original BRIEF pattern in Figure 2.

In summary, the rotationally symmetric pattern (RS-BRIEF) is generated by rotating the two sets of seeded locations, L_{S1} and L_{D1} . To calculate descriptors with RS-BRIEF pattern, the operations of rotating test locations can be reduced to changing the order of these locations or shifting the generated descriptor. And consequently it could be much more hardware friendly than original BRIEF descriptors by dramatically reducing the computation without introducing extra memory footprint.

3 eSLAM ARCHITECTURE

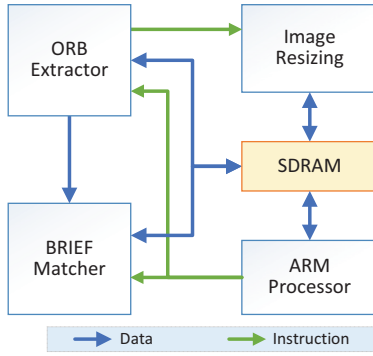


Figure 3: Overall architecture of eSLAM.

The overall architecture of the proposed ORB-based visual SLAM accelerator, eSLAM, is shown in Figure 3. It is partially accelerated on programmable logic of FPGA and hosted by an ARM processor. The ORB Extractor and the BRIEF Matcher are implemented to accelerate feature extraction and matching, which account for over 90% of the runtime on general computing platforms. And the Image Resizing module is adopted to generate image pyramids layer by layer for the ORB Extractor. When the ORB Extractor is processing one layer, the Image Resizing module applies nearest neighbor downsampling on the same layer to generate the next layer until the whole image pyramid is processed. The ARM processor performs pose estimation, pose optimization as well as map updating.

3.1 ORB Extractor

The ORB Extractor aims to extract ORB features from images. It reads data from SDRAM via AXI bus, and computes the ORB features with a local cache. After feature extraction is finished, it sends the result back to SDRAM and the descriptors of the features to the BRIEF Matcher. The original workflow of ORB feature extraction could be summarized as follows:

- (1) **Detecting** keypoints from the input image. Assuming that M keypoints are detected.
- (2) **Filtering** the keypoints. After filtering, only the N keypoints with the best Harris scores are kept, where $N < M$.

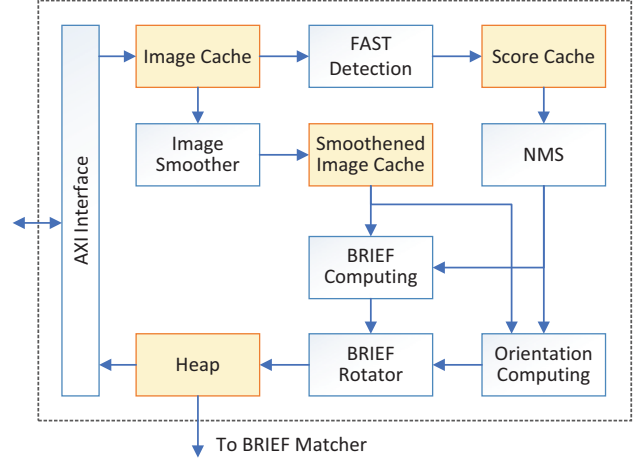


Figure 4: Architecture of the ORB Extractor.

- (3) **Computing** descriptors for the remained N keypoints.

Obviously there are two major problems when implementing the original workflow on hardware platforms. Firstly, the Detecting and Filtering procedures could be executed in parallel while the descriptors Computing procedure has to be idled until the Filtering is finished. Furthermore, it requires amount of on-chip cache to store the intermediate data when Computing the descriptors. In order to improve the computation throughput and reduce the memory consumption, the workflow of ORB feature extracting is rescheduled as a streaming manner as follows:

- (1) **Detecting** keypoints from the input image. Assuming that M keypoints are detected.
- (2) **Computing** descriptors for the detected M keypoints.
- (3) **Filtering** and reserving N features with the best Harris scores.

After rescheduling, the descriptors Computing procedure is executed before Filtering procedure so that they could run simultaneously and be pipelined for the streaming keypoints. Compared with the original workflow, there are $M - N$ extra keypoints calculated which will introduce some overheads but the latency has been optimized significantly due to the eliminated idle states. Moreover, the required on-chip cache is also reduced dramatically according to the streaming processing manner.

The detailed architecture of the ORB Extractor is shown in Figure 4. It is connected to AXI Interface and includes a FAST Detection module, a Image Smoother, an NMS (non-maximum suppression) module, a BRIEF Computing module, an Orientation Computing module, a BRIEF Rotator, a Heap and Caches (Image Cache, Score Cache and Smoothed Image Cache). The details of these modules are demonstrated as follows:

AXI Interface: The AXI Interface supports accessing SDRAM via AXI bus. The input image is read from SDRAM via AXI bus and stored in Image Cache while the computation results stored in the Heap are written back to SDRAM.

FAST Detection: The FAST Detection module takes a 7×7 pixels patch from the Image Cache as input. It detects FAST keypoint on this pixels patch and computes Harris corner score for each keypoint. If a FAST keypoint is detected, the corresponding Harris score is written into Score Cache.

Image Smoother: This module applies Gaussian blur operations on the 7×7 pixels patch of the original image for smoothing. Then the smoothed image is utilized for calculating descriptors and orientations of features.

NMS: The NMS module applies non-maximum suppression on the results of the FAST Detection module. It removes FAST keypoints that are too close to each other, and only reserves the one with maximum Harris score in any 3×3 pixels patch.

Orientation Computing: This module determines the orientation of each feature. The orientation is defined as the vector from the center of the feature to the mass center of the circular patch. The position (u, v) of the mass center is defined as:

$$\begin{aligned} u &= \frac{\sum_{(x,y) \in C} (I(x,y) \cdot x)}{\sum_{(x,y) \in C} I(x,y)} \\ v &= \frac{\sum_{(x,y) \in C} (I(x,y) \cdot y)}{\sum_{(x,y) \in C} I(x,y)} \end{aligned} \quad (3)$$

where C refers to the circular patch and $I(x, y)$ refers to the intensity of the pixel located at (x, y) . The Orientation Computing module builds a lookup table to determine the orientation from v/u and the signs of u and v . Since the pattern of the test locations is 32-fold rotationally symmetric, the feature orientations are discretized and represented by an integral label ranged from 0 to 31, where 0 represents 0 degree, 1 represents 11.25 degrees, 2 represents 22.5 degrees, etc.

BRIEF Computing: The BRIEF Computing module takes circular patches of smoothed pixels to calculate descriptors for features. The test locations it uses to generate descriptors follow the rotationally symmetric pattern we proposed.

BRIEF Rotator: The BRIEF Rotator shifts the descriptor according to the feature orientation, which provides the same results as rotating the test locations of RS-BRIEF. Assuming that the feature orientation is n , the BRIEF Rotator moves the $8 \times n$ bits from the beginning of the descriptor to the end.

Heap: The Heap is created to store and filter the descriptors, coordinates and Harris scores of features. To filter out some of the superfluous features, a max-heap structure is utilized to guarantee that only the 1024 features with the best Harris scores are reserved. Once the feature extraction is finished and stored in the heap, the descriptors and coordinates are sent to SDRAM through AXI Interface, and the descriptors are also delivered to the BRIEF Matcher.

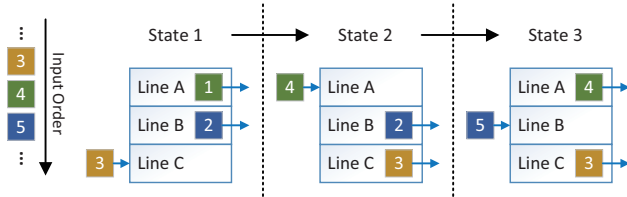


Figure 5: I/O mechanism of Image Cache. Line A, B and C refers to the 3 cache lines of Image Cache. Each square represents 8 columns of pixels.

Cache: There are 3 caches in ORB Extractor including the Image Cache storing pixels of the input image, the Score Cache storing the Harris scores of the keypoints, and the Smoothened Image Cache storing the smoothed image. These caches are designed

by a manner of “ping-pong mechanism” so that the streaming data could be processed simultaneously. The Image Cache is taken as an example to explain the data I/O mechanism. The Image Cache consists of 3 cache lines, each of which stores 8 columns of image pixels. As shown in Figure 5, the 3 cache lines receive input data by turns. The data I/O of the cache lines is controlled by a finite-state machine (FSM). The FSM is initialized by pre-storing 16 columns of pixels in cache line A and B. For each FSM state, one cache line receives input data while the other two send the data for output.

3.2 BRIEF Matcher

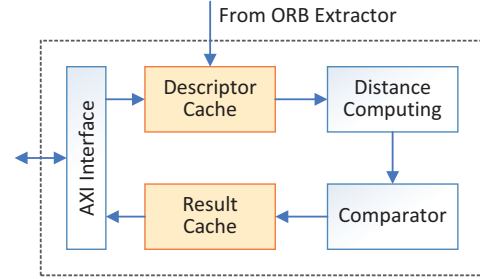


Figure 6: Architecture of the BRIEF Matcher.

In the BRIEF Matcher module, the features extracted from the current frame is compared with the map points of the global map. The features descriptors are obtained from the ORB Extractor, and the descriptors of global map are from SDRAM via AXI bus. The matching results are sent back to SDRAM at last.

The architecture of the BRIEF Matcher is shown in Figure 6. It is connected to AXI interface and includes a Descriptor Cache, a Distance Computing module, a Comparator and a Result Cache. The matching procedure starts following the ORB extraction. Assuming 2 sets of descriptors $D_A(DS_1, DS_2, \dots, DS_n)$ and $D_B(DD_1, DD_2, \dots, DD_m)$ have been pre-stored in Descriptor Cache, where D_A is the descriptors obtained from current frame, and D_B is the descriptors of the map points in the global map. For each descriptor DS_i in D_A , the Distance Computing module calculates the Hamming distances between DS_i and each descriptor DD_j in D_B . With the calculated Hamming distances $HD(H_{i1}, H_{i2}, \dots, H_{im})$, the Comparator searches through HD and finds the minimum value to determine the matching result and stores them into the Result Cache.

3.3 Parallelizing Mechanism

Since eSLAM is a heterogeneous system with the ARM processor as the host controller and FPGA as the acceleration modules, the parallelizing mechanism is critical to improve the computation throughput. The utilized parallelized pipeline is shown in Figure 7. For normal frames processing, while the ARM processor is performing pose estimation and pose optimization, the ORB Extractor and BRIEF Matcher are fired up to do feature extraction and feature matching for the next frame. However, it is different to process key frames because map updating is executed on the ARM processor after pose estimation and pose optimization. The ORB Extractor performs feature extraction on FPGA in parallel with the ARM processor, but the BRIEF Matcher would not start to work until map updating is finished.

With the parallelizing mechanism above, the several stages could be performed efficiently in pipeline. For normal frames, feature

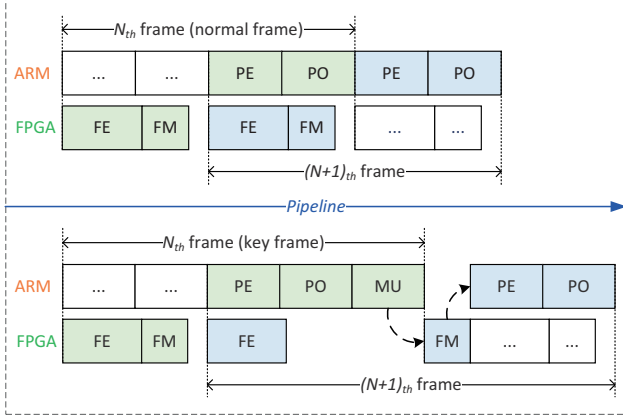


Figure 7: Parallelized pipeline of normal frame (upper) and key frame (lower), where FE refers to feature extraction, FM refers to feature matching, PE refers to pose estimation, PO refers to pose optimization and MU refers to map updating.

extraction and matching runs in parallel with pose estimation and optimization. And for key frames, feature extraction runs in parallel with pose estimation and optimization. These parallel processing manners could improve the computing throughput significantly.

4 EXPERIMENTAL RESULTS

4.1 Experimental Setup

Hardware Implementation: The proposed eSLAM system is implemented on Xilinx Zynq XCZ7045 SoC [12], which integrates an ARM Cortex-A9 processor and FPGA resources. The clock frequency of the ARM processor is 767 MHz, and the clock of accelerating modules is 100 MHz. The resource utilization of the proposed system is shown in Table 1. Since only about 1/4 resources are utilized on XCZ7045, it is possible to prototype them onto SoCs with less resources and lower price, such as XCZ7030/XCZ7020.

Table 1: The FPGA resources utilization of eSLAM.

	LUT	FF	DSP	BRAM
Utilization	56954 (26.0%)	67809 (15.5%)	111 (12.3%)	78 (14.3%)

Dataset: The proposed eSLAM is evaluated on TUM dataset [10]. It contains RGB images along with depth information and is widely used in visual SLAM community. The image resolution is 640×480 . Five different sequences in the dataset, *fr1/xyz*, *fr1/desk*, *fr1/room*, *fr2/xyz* and *fr2/rpy* are used for evaluation. Each sequence contains a ground truth trajectory that is obtained by a high-accuracy motion-capture system.

4.2 Accuracy Analysis

The accuracy of the visual SLAM system is measured by trajectory error which means the difference between the ground truth trajectory and the estimated trajectory. As shown in Figure 8, the average trajectory error is compared with the original ORB based SLAM implementation on the five sequences from TUM dataset.

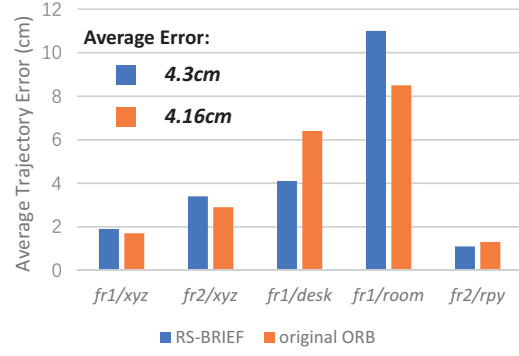


Figure 8: Average trajectory error of the SLAM implementation with RS-BRIEF compared with the original ORB on TUM dataset.

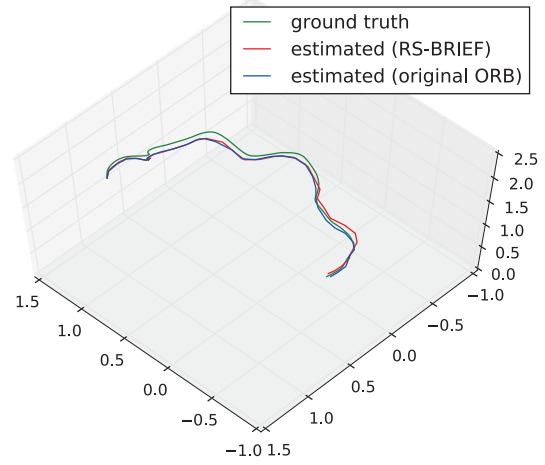


Figure 9: Estimated trajectory of the RS-BRIEF descriptor and original ORB descriptor based SLAM implementations, compared with the ground truth trajectory on *fr1/desk*.

For *fr1/xyz*, *fr1/room*, and *fr2/xyz* sequence, the implementation with original ORB has a better accuracy than with RS-BRIEF descriptor. However, the implementation with RS-BRIEF descriptor could have a better accuracy than with original ORB when evaluated on *fr1/desk* and *fr2/rpy* sequence. Among the five sequences, the total average error of RS-BRIEF based implementation is about 4.3 cm, and the original ORB based implementation is about 4.16 cm, which indicates that the accuracy of RS-BRIEF descriptor is comparable to the original descriptor.

Meanwhile, the trajectories estimated by the RS-BRIEF based implementation and the original ORB based implementation are also compared with the ground truth trajectory on *fr1/desk* sequence and visualized in Figure 9. Aiming to display the trajectories clearly, only a piece of them are selected as shown in Figure 9.

4.3 Performance Evaluation

The performance of the proposed eSLAM system is compared with the software implementations on the integrated ARM Cortex-A9 processor of XCZ7045 SoC and an Intel i7-4700mq processor [9]. The runtime comparison is shown in Table 2. Accelerated by ORB

Table 2: Detailed runtime breakdown of eSLAM compared with software implementations on ARM processor and Intel i7 CPU.

	eSLAM	ARM	Intel i7
Feature Extraction	9.1 ms	291.6 ms	32.5 ms
Feature Matching	4.0 ms	246.2 ms	19.7 ms
Pose Estimation	9.2 ms		0.9 ms
Pose Optimization	8.7 ms		0.5 ms
Map Updating	9.9 ms		1.2 ms

Extractor and BRIEF Matcher, the latency of feature extraction and matching procedure in eSLAM is reduced to 9.1 ms and 4 ms, respectively. Compared with Intel CPU and ARM, eSLAM could achieve 3.6× and 32× speedup in feature extraction, 4.9× and 61.6× speedup in feature matching.

Table 3 compares the average runtime per frame, the frame rate, the energy consumed per frame, the power consumption of eSLAM with the ARM processor and the Intel CPU. For normal frames, eSLAM performs feature extraction (FE) and matching (FM) simultaneously with pose estimation (PE) and optimization (PO). The average runtime is the sum of processing time of PE and PO, 17.9 ms. For key frames, FE is performed simultaneously with PE. eSLAM’s average runtime time is 31.8 ms, which is the sum of processing time of FM, PE, PO and MU. Compared with the ARM processor, eSLAM achieves about 17.8× speedup when processing key frames and 31× speedup for normal frames. Compared with the Intel i7 processor, it could achieve 1.7× to 3× speedups.

In terms of energy consumption, the proposed eSLAM also shows great advantage compared with the ARM and Intel CPU. Although the power consumption of eSLAM is increased by about 23% compared with the ARM processor due to the additional FPGA accelerating modules, the energy consumed per frame is still reduced by 14× to 25× depending on the key frame rate. Compared with the Intel i7 processor, the energy consumption is reduced by 41× to 71×.

4.4 Discussions

As shown in Table 3, the key frame rate of eSLAM is 31.45 fps, and the normal frame rate is 55.87 fps, which is much less than 171 fps which is achieved by Navion [11]. This gap is mainly because of the adopted different algorithms. Navion adopts the optical-flow method while only keypoints are detected but descriptors calculation and feature matching are not required. However, the adopted feature-based approach in eSLAM is much more robust in many scenarios where optical-flow methods may fail. Because the optical-flow methods are only available with two basic assumptions: constant illumination and small motions/displacements existed [5].

Compared with the ORB extractor implemented on FPGA in [4], the ORB extractor in eSLAM has deployed hardware-friendly optimization, such as RS-BRIEF and workflow rescheduling. Hence, the latency of feature extraction in eSLAM is approximately 39% less than the latency of [4], even if 48% more pixels are processed in eSLAM because of the involved extra two layers in the image pyramid.

Table 3: Frame rate and energy efficiency comparison results, where “N-frame” represents the normal frame, and “K-frame” represents the key frame.

		ARM	Intel i7	eSLAM
Runtime	N-frame	555.7 ms	53.6 ms	17.9 ms
	K-frame	565.6 ms	54.8 ms	31.8 ms
Frame Rate	N-frame	1.8 fps	18.66 fps	55.87 fps
	K-frame	1.77 fps	18.25 fps	31.45 fps
Power		1.574 W	47 W	1.936 W
Energy per Frame	N-frame	875 mJ	2519 mJ	35 mJ
	K-frame	890 mJ	2575 mJ	62 mJ

5 CONCLUSIONS

In this paper, a heterogeneous ORB-based visual SLAM system, eSLAM, is proposed for energy-efficient and real-time applications and evaluated on Zynq platforms. The ORB algorithm is first reformulated as a rotationally symmetric pattern for hardware-friendly implementation. Meanwhile, the most time-consuming stages, i.e., feature extraction and matching, are accelerated on FPGA to reduce the latency significantly. The eSLAM is also designed as a pipelined manner to further improve the throughput and reduce the memory footprint. The evaluation results on TUM dataset have shown eSLAM could achieve 1.7× to 3× speedup in frame rate, and 41× to 71× improvement in energy efficiency when compared with the Intel i7 CPU. Compared with the ARM processor, eSLAM could achieve 17.8× to 31× speedup in frame rate, and 14× to 25× improvement in energy efficiency.

REFERENCES

- [1] Michael Calonder, Vincent Lepetit, Christoph Strecha, and Pascal Fua. 2010. BRIEF: binary robust independent elementary features. In *Proceedings of European Conference on Computer Vision (ECCV)*. 778–792.
- [2] J. Cong, B. Grigorian, G. Reinman, and M. Vitanza. 2011. Accelerating vision and navigation applications on a customizable platform. In *Proceedings of IEEE International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. 25–32.
- [3] Hugh Durrant-Whyte and Tim Bailey. 2006. Simultaneous localization and mapping: part I. *IEEE robotics & automation magazine* 13, 2 (2006), 99–110.
- [4] Weikang Fang, Yanjun Zhang, Bo Yu, and Shaoshan Liu. 2017. FPGA-based ORB Feature Extraction for Real-Time Visual SLAM. In *Proceedings of International Conference on Field Programmable Technology (FPT)*. 275–278.
- [5] David Fleet and Yair Weiss. 2006. Optical flow estimation. In *Handbook of mathematical models in computer vision*. Springer, 237–257.
- [6] Mengyuan Gu, Kaiyuan Guo, Wenqiang Wang, Yu Wang, and Huazhong Yang. 2015. An FPGA-based real-time simultaneous localization and mapping system. In *Proceedings of International Conference on Field Programmable Technology (FPT)*. 200–203.
- [7] Jorge J. Moré. 1978. The Levenberg-Marquardt algorithm: Implementation and theory. *Lecture Notes in Mathematics* 630 (1978), 105–116.
- [8] Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary R. Bradski. 2012. ORB: an efficient alternative to SIFT or SURF. In *Proceedings of International Conference on Computer Vision (ICCV)*. 2564–2571.
- [9] Intel Chip’s Specifications. 2017. Intel Core i7-4700MQ Processor, 22nm. (2017). <https://ark.intel.com/content/www/us/en/ark/products/75117/intel-core-i7-4700mq-processor-6m-cache-up-to-3-40-ghz.html>.
- [10] Jürgen Sturm, Nikolas Engelhard, Felix Endres, Wolfram Burgard, and Daniel Cremers. 2012. A benchmark for the evaluation of RGB-D SLAM systems. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*. 573–580.
- [11] Amr Suleiman, Zhengdong Zhang, Luca Carlone, Sertac Karaman, and Vivienne Sze. 2019. Navion: A 2mW Fully Integrated Real-Time Visual-Inertial Odometry Accelerator for Autonomous Navigation of Nano Drones. *IEEE Journal of Solid-State Circuits (JSSC)* (2019), 1–14.
- [12] Xilinx. 2018. Zynq-7000 SoC. (2018). <https://www.xilinx.com/products/silicon-devices/soc/zynq-7000.html>.